Task 03: GitHub Dorking & Credential Leaks

During this task, I explored how exposed Git repositories can leak sensitive information. I used OWASP Juice Shop, an intentionally vulnerable application, to understand the concept safely.

Normally, a .git directory contains the full history of the project, including previous versions of files, configuration settings, and sometimes credentials. If this directory is accessible via a web server, an attacker could download it and reconstruct the repository to find sensitive data.

Setting up Git repository using python server

To simulate this, I set up a local web server with a test Git repository containing a sample password.

```
┌──(hyena㉿hyena)-[~/Downloads/GitHacker/githacker/vulnerable-site]
└─$ python3 -m http.server 8000
Serving HTTP on 0.0.0.0 port 8000 (http://0.0.0.0:8000/) ...
127.0.0.1 - - [07/Mar/2026 20:47:23] "GET /.git/ HTTP/1.1" 200 -
127.0.0.1 - - [07/Mar/2026 20:47:24] code 404, message File not found
127.0.0.1 - - [07/Mar/2026 20:47:24] "GET /favicon.ico HTTP/1.1" 404 -
127.0.0.1 - - [07/Mar/2026 20:47:31] "GET /.git/refs/ HTTP/1.1" 200 -
127.0.0.1 - - [07/Mar/2026 20:47:33] "GET /.git/refs/tags/ HTTP/1.1" 200 -
127.0.0.1 - - [07/Mar/2026 20:50:22] "GET /.git/objects/ HTTP/1.1" 200 -
127.0.0.1 - - [07/Mar/2026 20:50:24] "GET /.git/objects/info/ HTTP/1.1" 200 -
127.0.0.1 - - [07/Mar/2026 20:50:28] "GET /.git/config HTTP/1.1" 200 -
127.0.0.1 - - [07/Mar/2026 20:50:31] "GET /.git/description HTTP/1.1" 200 -
127.0.0.1 - - [07/Mar/2026 20:50:32] "GET /.git/index HTTP/1.1" 200 -
127.0.0.1 - - [07/Mar/2026 20:54:59] "GET /.git/HEAD HTTP/1.1" 200 -
127.0.0.1 - - [07/Mar/2026 20:54:59] code 404, message File not found
127.0.0.1 - - [07/Mar/2026 20:54:59] "GET /.git/logs/HEAD HTTP/1.1" 404 -
127.0.0.1 - - [07/Mar/2026 20:54:59] "HEAD /.git/HEAD HTTP/1.1" 200 -
127.0.0.1 - - [07/Mar/2026 20:54:59] "GET /.git/ HTTP/1.1" 200 -
127.0.0.1 - - [07/Mar/2026 20:54:59] "GET /.git/HEAD HTTP/1.1" 200 -
127.0.0.1 - - [07/Mar/2026 20:54:59] "GET /.git/description HTTP/1.1" 200 -
127.0.0.1 - - [07/Mar/2026 20:54:59] code 404, message File not found
127.0.0.1 - - [07/Mar/2026 20:54:59] "GET /.git/FETCH_HEAD HTTP/1.1" 404 -
127.0.0.1 - - [07/Mar/2026 20:54:59] "GET /.git/COMMIT_EDITMSG HTTP/1.1" 200 -
127.0.0.1 - - [07/Mar/2026 20:54:59] "GET /.git/index HTTP/1.1" 200 -
127.0.0.1 - - [07/Mar/2026 20:54:59] code 404, message File not found
127.0.0.1 - - [07/Mar/2026 20:54:59] "GET /.git/info/exclude HTTP/1.1" 200 -
127.0.0.1 - - [07/Mar/2026 20:54:59] "GET /.git/logs/HEAD HTTP/1.1" 404 -
127.0.0.1 - - [07/Mar/2026 20:54:59] code 404, message File not found
127.0.0.1 - - [07/Mar/2026 20:54:59] code 404, message File not found
127.0.0.1 - - [07/Mar/2026 20:54:59] "GET /.git/logs/refs/stash HTTP/1.1" 404 -
127.0.0.1 - - [07/Mar/2026 20:54:59] "GET /.git/logs/refs/remotes/origin/HEAD HTTP/1.1" 404 -
127.0.0.1 - - [07/Mar/2026 20:54:59] code 404, message File not found
127.0.0.1 - - [07/Mar/2026 20:54:59] "GET /.git/ORIG_HEAD HTTP/1.1" 404 -
127.0.0.1 - - [07/Mar/2026 20:54:59] code 404, message File not found
127.0.0.1 - - [07/Mar/2026 20:54:59] "GET /.git/packed-refs HTTP/1.1" 404 -
127.0.0.1 - - [07/Mar/2026 20:54:59] code 404, message File not found
127.0.0.1 - - [07/Mar/2026 20:54:59] "GET /.git/refs/stash HTTP/1.1" 404 -
127.0.0.1 - - [07/Mar/2026 20:54:59] code 404, message File not found
127.0.0.1 - - [07/Mar/2026 20:54:59] "GET /.git/refs/remotes/origin/HEAD HTTP/1.1" 404 -
127.0.0.1 - - [07/Mar/2026 20:54:59] code 404, message File not found
127.0.0.1 - - [07/Mar/2026 20:54:59] "GET /.git/objects/info/alternates HTTP/1.1" 404 -
127.0.0.1 - - [07/Mar/2026 20:54:59] code 404, message File not found
127.0.0.1 - - [07/Mar/2026 20:54:59] "GET /.git/objects/info/http-alternates HTTP/1.1" 404 -
127.0.0.1 - - [07/Mar/2026 20:54:59] code 404, message File not found
127.0.0.1 - - [07/Mar/2026 20:54:59] "GET /.git/objects/info/packs HTTP/1.1" 404 -
127.0.0.1 - - [07/Mar/2026 20:54:59] code 404, message File not found
127.0.0.1 - - [07/Mar/2026 20:54:59] "GET /.git/refs/tags/v0.0.1 HTTP/1.1" 404 -
127.0.0.1 - - [07/Mar/2026 20:54:59] code 404, message File not found
127.0.0.1 - - [07/Mar/2026 20:54:59] "GET /.git/refs/tags/0.0.1 HTTP/1.1" 404 -
```

Using GitHacker, I was able to download and reconstruct the repository locally, then examine the commit history to see previous versions and stored credentials. This process clearly demonstrates

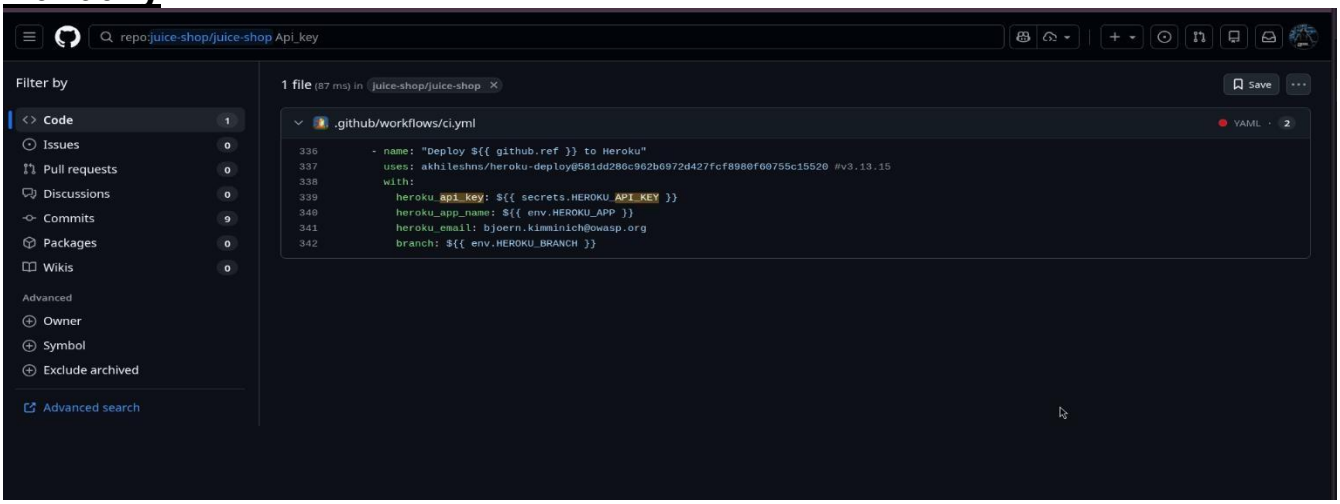
how careless configuration can expose passwords, API keys, or other sensitive information in previous commits.

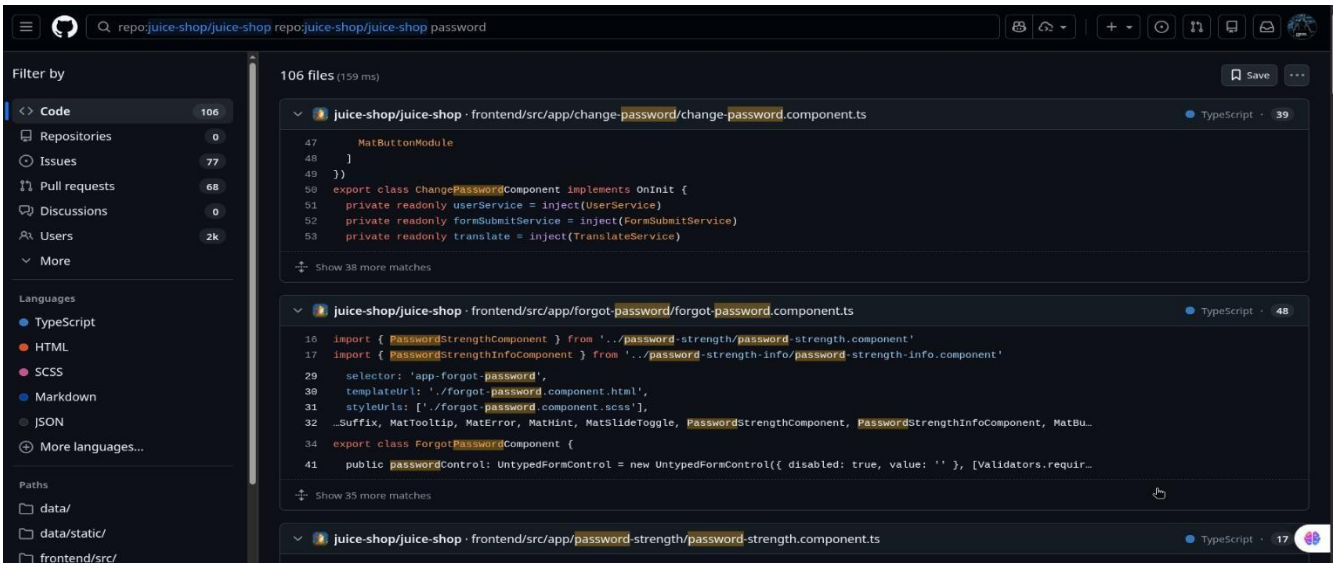
```
┌──(venv)─(hyena㉿hyena)-[~/Downloads/GitHacker]
└─$ python -m githacker --url http://localhost:8000/.git/ --output-folder recovered_repo
2026-03-07 20:54:59 INFO 1 urls to be exploited
2026-03-07 20:54:59 INFO Exploiting http://localhost:8000/.git/ into recovered_repo/4eb233cad7356878222ee5009629383d
2026-03-07 20:54:59 INFO Downloading basic files...
2026-03-07 20:54:59 INFO [23 bytes] 200 .git/HEAD
2026-03-07 20:54:59 ERROR FileExistsError(17, 'File exists')
2026-03-07 20:54:59 ERROR FileExistsError(17, 'File exists')
2026-03-07 20:54:59 ERROR [335 bytes] 404 .git/FETCH_HEAD
2026-03-07 20:54:59 INFO [73 bytes] 200 .git/description
2026-03-07 20:54:59 ERROR FileExistsError(17, 'File exists')
2026-03-07 20:54:59 ERROR FileExistsError(17, 'File exists')
2026-03-07 20:54:59 INFO [65 bytes] 200 .git/index
2026-03-07 20:54:59 INFO [29 bytes] 200 .git/COMMIT_EDITMSG
2026-03-07 20:54:59 INFO [240 bytes] 200 .git/info/exclude
2026-03-07 20:54:59 ERROR FileExistsError(17, 'File exists')
2026-03-07 20:54:59 ERROR [335 bytes] 404 .git/logs/HEAD
2026-03-07 20:54:59 ERROR FileExistsError(17, 'File exists')
2026-03-07 20:54:59 ERROR [335 bytes] 404 .git/logs/refs/stash
2026-03-07 20:54:59 ERROR [335 bytes] 404 .git/logs/refs/remotes/origin/HEAD
2026-03-07 20:54:59 ERROR FileExistsError(17, 'File exists')
2026-03-07 20:54:59 ERROR [335 bytes] 404 .git/ORIG_HEAD
2026-03-07 20:54:59 ERROR FileExistsError(17, 'File exists')
2026-03-07 20:54:59 ERROR [335 bytes] 404 .git/packed-refs
2026-03-07 20:54:59 ERROR [335 bytes] 404 .git/refs/stash
2026-03-07 20:54:59 ERROR [335 bytes] 404 .git/refs/remotes/origin/HEAD
2026-03-07 20:54:59 ERROR FileExistsError(17, 'File exists')
2026-03-07 20:54:59 ERROR [335 bytes] 404 .git/objects/info/alternates
2026-03-07 20:54:59 ERROR [335 bytes] 404 .git/objects/info/http-alternates
2026-03-07 20:54:59 ERROR FileExistsError(17, 'File exists')
2026-03-07 20:54:59 ERROR [335 bytes] 404 .git/objects/info/packs
2026-03-07 20:54:59 ERROR [335 bytes] 404 .git/refs/tags/v0.0.1
2026-03-07 20:54:59 ERROR FileExistsError(17, 'File exists')
2026-03-07 20:54:59 ERROR [335 bytes] 404 .git/refs/tags/0.0.1
2026-03-07 20:54:59 ERROR FileExistsError(17, 'File exists')
2026-03-07 20:54:59 ERROR [335 bytes] 404 .git/logs/refs/heads/daily
```

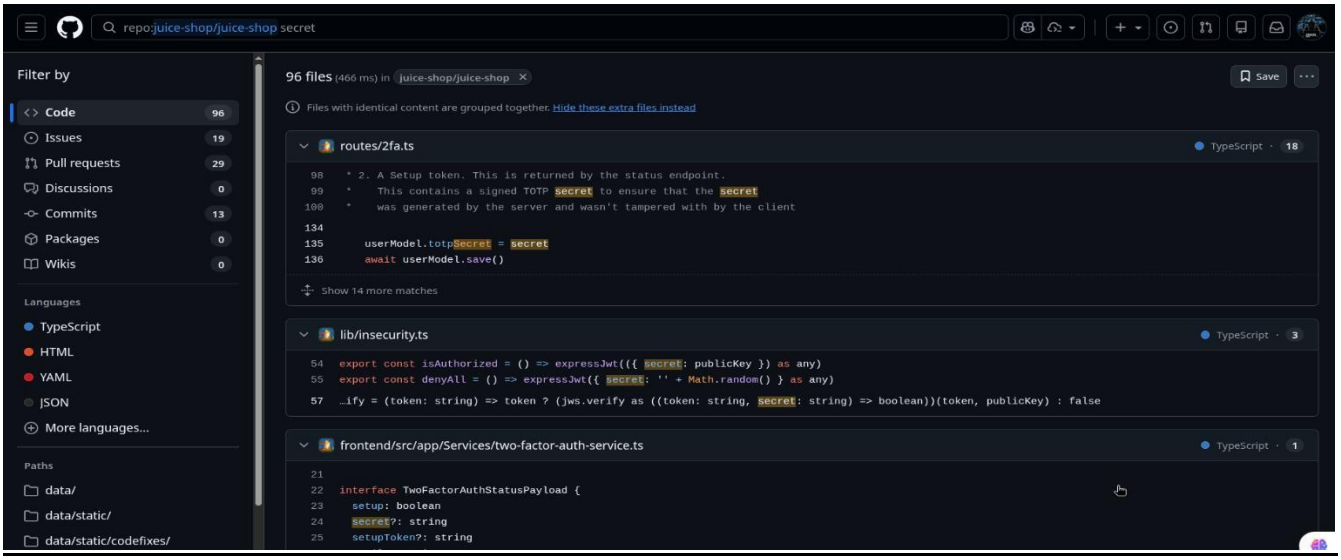
This exercise shows the importance of properly securing .git directories on web servers and following best deployment practices to prevent accidental exposure of sensitive data.

```
┌──(hyena㉿hyena)-[~/…/4eb233cad7356878222ee5009629383d/.git/logs/refs]
└─$ ls
heads  remotes

┌──(hyena㉿hyena)-[~/…/4eb233cad7356878222ee5009629383d/.git/logs/refs]
└─$ cd ../../

┌──(hyena㉿hyena)-[~/…/GitHacker/recovered_repo/4eb233cad7356878222ee5009629383d/.git]
└─$ ls
COMMIT_EDITMSG  config  description  HEAD  hooks  info  logs  objects  refs

┌──(hyena㉿hyena)-[~/…/GitHacker/recovered_repo/4eb233cad7356878222ee5009629383d/.git]
└─$ cd ../../
```

Manually


repo:juice-shop/juice-shop Api_key
Filter by
Code 1
Issues 0
Pull requests 0
Discussions 0
Commits 9
Packages 0
Wikis 0
Advanced
Owner
Symbol
Exclude archived
Advanced search
1 file (87 ms) in juice-shop/juice-shop
Save
.github/workflows/ci.yml
YAML · 2
336 - name: "Deploy ${{ github.ref }} to Heroku"
337 uses: akhileshns/heroku-deploy@581dd280c962b0972d427fcf8980f60755c15520 #v3.13.15
338 with:
339 heroku_api_key: ${{ secrets.HEROKU_API_KEY }}
340 heroku_app_name: ${{ env.HEROKU_APP }}
341 heroku_email: bjoern.kimminich@owasp.org
342 branch: ${{ env.HEROKU_BRANCH }}


repo:juice-shop/juice-shop repo:juice-shop/juice-shop password
Filter by
Code 106
Repositories 0
Issues 77
Pull requests 68
Discussions 0
Users 2k
More
Languages
TypeScript
HTML
SCSS
Markdown
JSON
More languages...
Paths
data/
data/static/
frontend/src/
106 files (159 ms)
Save
juice-shop/juice-shop · frontend/src/app/change-password/change-password.component.ts
TypeScript · 39
47 MatButtonModule
48]
49 })
50 export class ChangePasswordComponent implements OnInit {
51 private readonly userService = inject(UserService)
52 private readonly formSubmitService = inject(FormSubmitService)
53 private readonly translate = inject(TranslateService)
Show 38 more matches
juice-shop/juice-shop · frontend/src/app/forgot-password/forgot-password.component.ts
TypeScript · 48
16 import { PasswordStrengthComponent } from '../password-strength/password-strength.component'
17 import { PasswordStrengthInfoComponent } from '../password-strength-info/password-strength-info.component'
29 selector: 'app-forgot-password',
30 templateUrl: './forgot-password.component.html',
31 styleUrls: ['./forgot-password.component.scss'],
32 …Suffix, MatTooltip, MatError, MatHint, MatSlideToggle, PasswordStrengthComponent, PasswordStrengthInfoComponent, MatBu…
34 export class ForgotPasswordComponent {
41 public passwordControl: UntypedFormControl = new UntypedFormControl({ disabled: true, value: '' }, [Validators.requir…
Show 35 more matches
juice-shop/juice-shop · frontend/src/app/password-strength/password-strength.component.ts
TypeScript · 17


repo:juice-shop/juice-shop secret
Filter by
Code 96
Issues 19
Pull requests 29
Discussions 0
Commits 13
Packages 0
Wikis 0
Languages
TypeScript
HTML
YAML
JSON
More languages...
Paths
data/
data/static/
data/static/codefixes/
96 files (466 ms) in juice-shop/juice-shop
Save
Files with identical content are grouped together. Hide these extra files instead
routes/2fa.ts
TypeScript · 18
98 * 2. A Setup token. This is returned by the status endpoint.
99 * This contains a signed TOTP secret to ensure that the secret
100 * was generated by the server and wasn't tampered with by the client
134
135 userModel.totpSecret = secret
136 await userModel.save()
Show 14 more matches
lib/insecurity.ts
TypeScript · 3
54 export const isAuthorized = () => expressJwt(({ secret: publicKey }) as any)
55 export const denyAll = () => expressJwt({ secret: '' + Math.random() } as any)
57 …ify = (token: string) => token ? (jws.verify as ((token: string, secret: string) => boolean))(token, publicKey) : false
frontend/src/app/Services/two-factor-auth-service.ts
TypeScript · 1
21
22 interface TwoFactorAuthStatusPayload {
23 setup: boolean
24 secret?: string
25 setupToken?: string

Another common method used to find leaked credentials is GitHub dorking on GitHub, where researchers search public repositories using specific keywords that may reveal sensitive information. Developers sometimes accidentally upload configuration files or secrets when pushing code. Common search keywords include **password, passwd, api_key, secret, token, access_key, private_key, client_secret, db_password, AWS_ACCESS_KEY, authorization, .env, config, credentials, and database_ur**l. These terms often appear in configuration files, environment files, or scripts and may expose internal credentials or service keys. Security case studies published on platforms like Medium and InfoSec Write-ups show that using such keywords can sometimes reveal organization domains, internal accounts, and API tokens if repositories are publicly accessible.